

AI-Powered Outbound Calls (Arabic) — 4-Sprint Plan (1 Week per Sprint)

v2 — reordered for sprint-by-sprint testability

Team: 5 members, one per module **Duration:** 4 weeks total **Sequencing logic:** Foundational/independent work goes in Sprint 1. Anything with a real dependency is scheduled *after* the blocking task's sprint completes — and within each person's own track, tasks are ordered so nothing needs to be rebuilt on top of infrastructure that doesn't exist yet.

Changes from v1:

- 1. **Person B:** Swapped Sprint 2 ↔ Sprint 3. Retry logic needs the Celery/Redis queue to actually reschedule failed calls — building retry logic before the queue existed would have meant a throwaway implementation in Sprint 2, rebuilt properly in Sprint 3.
- 2. **Person E:** Documented a previously-undocumented dependency — the Sprint 4 chat widget integration is blocked by *both* Person C's RAG/GPT-4 work *and* Person D's access-control work (the RAG API likely requires an authenticated request once OAuth2 is live). Both resolve by end of Sprint 3, so no timing change was needed, just the documentation.
- 3. **Person D:** Moved logging setup (ELK/Grafana+Loki) from Sprint 2 into Sprint 1, paired with the schema work. This means centralized logging exists from day one, so everyone's Sprint 1 work (STT tests, dial tests, etc.) can be debugged against real logs instead of flying blind for the first week.

Dependency map (why the order is what it is)

Blocked task	Depends on	Owner of dependency	Resolved by
Retry logic + fallback transfer (Telephony)	Queue system — Celery/Redis (Telephony, same person)	Person B	End of Sprint 2
CRM/customer-flagged-list integration (Telephony)	Call logs schema (Backend)	Person D	End of Sprint 1
Dialog logic ↔ STT integration (Conversation)	Integrate Arabic STT (Speech)	Person A	End of Sprint 1

Blocked task	Depends on	Owner of dependency	Resolved by
RAG ↔ KB document storage (Conversation)	KB storage schema (Backend)	Person D	End of Sprint 1
Dashboard ↔ reporting API (Frontend)	Reporting pipeline (Backend)	Person D	End of Sprint 2
Chat widget ↔ RAG API (Frontend)	RAG pipeline for chatbot (Conversation)	Person C	End of Sprint 3
Chat widget ↔ RAG API (Frontend)	OAuth2 / access control (Backend) — RAG API likely requires auth	Person D	End of Sprint 3

Person A — Speech Processing (STT/TTS)

(unchanged — order confirmed correct)

Sprint	Tasks	Goal
Sprint 1	Set up Whisper API / self-hosted model · Test accuracy on Egyptian Arabic vs MSA samples	Working, validated Arabic STT pipeline — unblocks Conversation's dialog integration
Sprint 2	Select TTS provider and get Arabic voice samples · Generate dynamic greeting script playback	Natural-sounding Arabic TTS with dynamic call greetings
Sprint 3	Set up pydub/ffmpeg for call audio format handling	Reliable audio format conversion between telephony and STT/TTS
Sprint 4	Support full call-flow integration testing (Person E) · Tune STT/TTS based on real test-call results	Polish, bug fixes, accuracy tuning from integration feedback

Person B — Telephony & Call Orchestration

(REORDERED — Sprint 2 and 3 swapped)

Sprint	Tasks	Goal
Sprint 1	Create Twilio/Vonage account, get API keys, test outbound dial	Confirmed ability to place outbound calls programmatically
Sprint 2	Build queue system with Celery and Redis	Scheduled/queued outbound calling working end-to-end — gives retry logic something real to schedule against
Sprint 3	Build retry logic for no-answer/failed call · Build fallback-to-human-agent transfer	Resilient call handling — retries now hook into the real queue instead of a throwaway mechanism; live-agent escalation working
Sprint 4	Integrate CRM/customer-flagged-list API (depends on Backend's schema, done Sprint 1) · Support full call-flow integration testing	Scheduler pulls real follow-up lists from CRM; help validate full flow

Person C — Conversation/NLU & RAG

(unchanged — order confirmed correct)

Sprint	Tasks	Goal
Sprint 1	Build branching dialog tree (yes/no/uncertain)	Core dialog decision logic working on placeholder/mock STT input
Sprint 2	Set up vector DB (Pinecone) · Build document ingestion/chunking/embedding job	KB documents embedded and searchable
Sprint 3	Integrate GPT-4 Turbo for answer generation with citations · Integrate dialog logic with finalized STT output (depends on Person A, done Sprint 1)	RAG chatbot gives cited answers; dialog logic now uses real STT output
Sprint 4	Connect RAG retrieval pipeline to KB document storage (depends on Backend's KB schema, done Sprint 1) · Support integration testing	Full RAG pipeline live against real KB storage; help validate chatbot accuracy tests

Person D — Backend/Data & Reporting

(REORDERED — logging moved from Sprint 2 to Sprint 1)

Sprint	Tasks	Goal
Sprint 1	Design call logs schema (PostgreSQL/MongoDB) · Design KB storage schema · Set up logging (ELK or Grafana+Loki)	Schemas ready — unblocks Telephony's CRM integration and Conversation's RAG storage connection. Centralized logging live from day one so the whole team can debug against real logs, not blind guessing, starting in week 1
Sprint 2	Build First Call Resolutions report generator	Reporting pipeline live — unblocks Frontend's dashboard integration
Sprint 3	Implement OAuth2 / role-based access for KB data	Secure, auditable access to chatbot and internal data — this also becomes a second blocker for Frontend's Sprint 4 chat widget integration, since the RAG API will likely require authenticated requests
Sprint 4	Support integration testing · Fix any schema/reporting issues found during full-flow testing	System hardening, bug fixes from integration feedback

Note: Sprint 1 is now the heaviest sprint for Person D (3 tasks vs. 1 in Sprint 2). This is intentional — the schema work is the most load-bearing dependency in the whole plan, and now logging joins it since early debuggability benefits everyone. If Person D needs to lighten Sprint 1, logging is the one task here that could safely slide back to Sprint 2 without blocking anyone else.

Person E — Frontend/Dashboard

(unchanged timing — new dependency documented for Sprint 4)

Sprint	Tasks	Goal
Sprint 1	Build React dashboard shell	Base dashboard app structure ready
Sprint 2	Add charts for FCR rate/AHT/completion percent · Build embeddable chat UI component	Dashboard visuals and chat widget UI built (using mock data for now)
Sprint 3	Integrate FCR dashboard with backend reporting API (depends on Person D, done Sprint 2)	Dashboard shows live data instead of mocks
Sprint 4	Connect chat widget to RAG pipeline's answer-generation API (depends on Person C's RAG work <i>and</i> Person D's access-control work, both done Sprint 3 — remember to include an auth token in this API call, not just a raw request) · Write integration tests across full call flow · Write test suite for chatbot accuracy	Full system connected end-to-end; test coverage in place

Sprint-by-sprint team summary (updated)

Sprint 1 — Foundations, testable in isolation STT setup + accuracy testing, Twilio account + test dial, dialog tree on mock data, DB schemas + centralized logging, dashboard shell. Every person has something independently demoable by Friday — none of it requires another team member's work to exist first.

Sprint 2 — Core features, still mostly self-contained TTS + greetings, **queue system** (Celery/Redis), vector DB + document ingestion, FCR report generator, charts + chat UI (on mocks). Testable per person; Backend's report generator is the one piece other people will start depending on next sprint.

Sprint 3 — First real integrations Audio format handling, **retry logic + fallback** (now testable against the real queue), GPT-4 RAG generation + real STT-dialog integration, access control, dashboard-to-backend integration (first cross-team connection to test).

Sprint 4 — Final integrations, testing, and polish CRM integration, RAG-to-storage integration, chat-widget-to-RAG integration (now correctly documented as needing both RAG *and* auth), full end-to-end testing, bug fixing based on real integration results.

Suggested sprint ceremonies

- **Sprint planning:** Monday morning, 30 min — confirm the week's tasks and flag any blockers early
- **Daily standup:** 10 min async (Slack/Jira comment) or quick call — what you did, what's next, any blockers
- **Sprint review/demo:** Friday afternoon — each person shows their working piece
- **Sprint retro:** Friday, 15 min — what worked, what to improve for next sprint